

RESEARCH ARTICLE SUMMARY

SCIENCE OF SCIENCE

Who is using AI to code? Global diffusion and impact of generative AI

Simone Daniotti*, Johannes Wachs, Xiangnan Feng, Frank Neffke



Full article and list of author affiliations:
<https://doi.org/10.1126/science.adz9311>

INTRODUCTION: Generative artificial intelligence (genAI) has the potential to radically transform the nature and productivity of human labor, yet we lack systematic measurement of how fast genAI is diffusing and who benefits from it. In this work, we develop a method to detect AI-generated code at the individual level and apply it to more than 30 million code contributions by 160,097 software developers across six countries. We find that genAI adoption is rapid and global and that it boosts productivity and accelerates expansion into new technical domains—but only for senior-level developers. Early-career developers, by contrast, despite being the most frequent users of genAI, show no measurable benefits. Therefore, rather than closing skill gaps, genAI appears to be widening them.

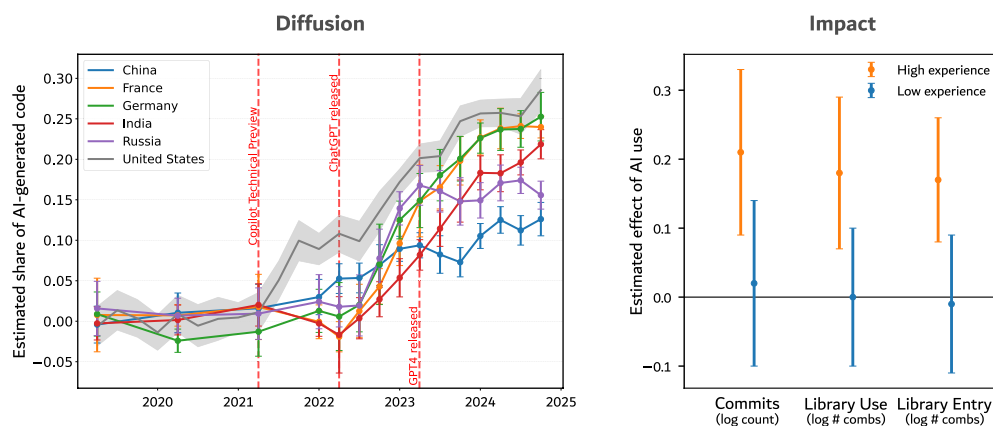
RATIONALE: We measure the uptake of genAI coding assistants in Python projects on GitHub, the world's largest collaboration platform for software development. We developed a neural network classifier that accurately determines whether a function—a self-contained unit of code—was generated by AI or written by a human. We trained this classifier on human-written functions expanded with synthetic equivalents created by chaining two large language models (LLMs): one model describing human code in natural language, which a second model uses to reimplement the function. We applied this classifier to more than 30 million commits from 160,097 developers across six countries (US, France, Germany, India, China, and Russia) between 2019 and 2024, charting genAI diffusion globally. For 100,097 US developers, we use fixed-effects models—comparing how the same developer's output changes with changes in their genAI usage—to estimate effects on the nature and volume of code produced.

RESULTS: GenAI tools have diffused rapidly, with sharp upticks coinciding with new generations of coding assistants. The US had an early yet narrowing lead: By the end of 2024, an estimated 29% of Python functions were produced with substantial AI support. France and Germany draw close (23 and 24%, respectively), and India caught up considerably, although China and Russia still lag. In the US sample, we find no gender difference in usage rates, but early-career developers use genAI more often compared with senior-level developers.

GenAI increases productivity—point estimates suggest a 3.6% boost in quarterly commit rates at current adoption levels. At the level of the US economy, this additional output translates conservatively into a yearly value of 23 billion to 38 billion USD, but the true value of genAI may be substantially larger. GenAI is also associated with increased experimentation with new software libraries, which suggests that it helps programmers expand into unfamiliar technical domains. Notably, however, these productivity and exploration gains accrue exclusively to senior-level developers. Despite their higher adoption rates, early-career developers show no significant benefits.

CONCLUSION: We show that genAI use can be accurately detected in the digital traces of software developers, revealing rapid global diffusion of AI coding tools. GenAI increases output and helps programmers expand into new domains—but only for senior-level developers. Early-career developers, despite being the most enthusiastic adopters, see no measurable gains. This leads to substantial uncertainty for young developers about their career development in the software sector. □

*Corresponding author. Email: daniotti@csh.ac.at Cite this article as S. Daniotti et al., *Science* 391, 831 (2026). DOI: [10.1126/science.adz9311](https://doi.org/10.1126/science.adz9311)



Global diffusion of genAI coding assistants and their impact. (Left) The share of AI-written Python functions (2019 to 2024) grows rapidly, but countries differ in their adoption rates. (Right) Comparing usage rates for the same developers at different points in time, genAI adoption is associated with increased productivity (commits), breadth of functionality (library use), and exploration of new functionality (library entry), but only for senior developers, whereas early-career developers do not derive any statistically significant benefits from using genAI.

SCIENCE OF SCIENCE

Who is using AI to code? Global diffusion and impact of generative AI

Simone Daniotti^{1,2*}, Johannes Wachs^{1,3,4},
Xiangnan Feng¹, Frank Neffke^{1,5}

Generative coding tools promise big productivity gains, but uneven uptake could widen skill and income gaps. We train a neural classifier to spot artificial intelligence (AI)–generated Python functions in more than 30 million GitHub commits by 160,097 software developers, tracking how fast, and where, these tools take hold. Currently, AI writes an estimated 29% of Python functions in the US—a shrinking lead over other countries. We estimate that quarterly output, measured in online code contributions, consequently increased by 3.6%. AI seems to benefit experienced, senior-level developers: They increased productivity and more readily expanded into new domains of software development. By contrast, early-career developers showed no significant benefits from AI adoption. This may widen skill gaps and reshape future career ladders in software development.

According to proponents, artificial intelligence (AI)—in particular generative AI (genAI)—will markedly increase our productivity and revolutionize the way that we work. For instance, genAI is expected to complement or substitute humans in an increasing set of tasks (1). This forces individuals, firms, and policy-makers to make important decisions about the use and regulation of genAI under major uncertainty. The stakes are high: GenAI has become widely accessible through tools such as ChatGPT or Claude, directly complements human thinking (2), and holds the potential of becoming a general-purpose technology that can solve a wide variety of problems (3).

Experimental and quasi-experimental evidence so far support the notion that genAI has transformative potential, showing that the use of genAI leads to increases in productivity and output of individual workers in a variety of jobs (1, 4–6). Surveys and data reported by large language model (LLM) owners suggest that these technologies are diffusing rapidly (7–9). Yet, estimates of the aggregate effect of AI on gross domestic product (GDP) and employment are often modest (10, 11), which suggests that we are far from having a clear picture of the overall impacts of AI.

We do know that there is substantial heterogeneity of adoption, which could lead to economic divergence. Although genAI use is widespread in the working-age population, self-reported adoption rates differ markedly across demographics, seniority, work experience, and sectors (8, 9). Evidence from job ads and firm websites suggests that the adoption of genAI varies across geography (12, 13). If genAI substantially raises productivity, any implied barriers to adoption will have important consequences for inequality within and across countries (14). Historically, however, macrolevel productivity effects of general-purpose technologies, such as steam engines, dynamos, and

computers, have taken long to materialize (4, 15–17). Together, this leads to substantial uncertainty about the impact of genAI today.

Resolving this uncertainty requires accurately determining adoption rates, intensity of use, and productivity effects at a global level. Surveys demonstrating demographic and sectoral heterogeneities in genAI adoption often focus on single countries (8, 9). Previous work comparing AI adoption in different countries using survey data has found evidence of differences within and between countries (18), but differences in sample weighting and analysis periods of the surveys limit our ability to directly compare observed rates. In the context of genAI, respondents may underreport usage, especially at work, to avoid judgment (19, 20). Nevertheless, surveys provide a valuable resource for understanding adoption patterns. Similarly, randomized controlled trials (RCTs) (1, 4–6, 21) and natural experiments (22–24) are indispensable because they measure causal effects of genAI adoption by design. However, they typically consider individuals as “treated” whenever they have access to genAI tools, without quantifying the extent to which treated individuals used genAI during the experiments. Moreover, surveys and experiments tend to observe individuals over short time periods, which limits our ability to know the dynamics of adoption and to observe effects of adoption that materialize more slowly.

To begin to address these gaps, we ask whether we can directly measure the adoption and intensity of use of genAI by individuals over time using machine learning instead of from self-reported information. If so, what do such measures tell us about the rate of adoption of genAI? Does this differ across countries and demographics? How does genAI affect the output that individuals produce, and how do individual characteristics such as experience moderate such effects?

To answer these questions, we study genAI use at a fine-grained level in one of its main domains of application, software development—an important and high-value sector (25, 26) that is particularly exposed to genAI (21, 22, 27, 28). To do so, we design and implement a machine learning classifier to identify code written with substantial AI assistance in more than 30 million software developer contributions, also known as commits, to open-source Python projects on GitHub. To train this classifier, we assemble a custom training set, combining existing sources with a procedure that generates synthetic training data. This allows us to analyze shifting patterns of AI-generated code at a granular level. We leverage this source of microdata to study how quickly the use of genAI in coding diffuses in six major countries, how this diffusion relates to demographic characteristics, and how it affects programming activity in a sample of 100,097 US software developers.

Detecting AI-generated code

To collect a large dataset of coding activity, we gather all commits by 100,097 US GitHub users to Python-based open-source projects, recursively cloning all GitHub directories related to each project. Next, we add commits from a random sample of 2000 developers per year (60,000 developers in total) for each of five other major countries in software development: China, France, Germany, India, and Russia. We then analyze these commits to assess the prevalence of AI-generated code (materials and methods).

Figure 1 describes how we classify these code contributions as either human or AI generated. We limit this analysis to blocks of code that represent functions to focus on a fine-grained, self-contained, yet substantive unit of code. We first construct a ground truth dataset (Fig. 1A), collecting Python functions that we are certain were written by a human programmer or developer. To do so, we take functions written in 2018 because they predate the release of capable genAI models. Because programming styles evolve over time, we add functions created in later years from the HumanEval datasets for the years 2022 and 2024. To add a dataset of similar functions but written by genAI, we apply a two-step procedure. First, for each human-written function, we ask one LLM to describe the function in English, specifying the type of input and output of the function. Second, we feed this

¹Complexity Science Hub, Vienna, Austria. ²Copernicus Institute of Sustainable Development, Utrecht University, Utrecht, Netherlands. ³Department of Network Science, Corvinus University of Budapest, Budapest, Hungary. ⁴Agglomeration, Networks, and Innovation Research Group, ELTE Centre for Economic and Regional Studies, Budapest, Hungary.

⁵Transforming Economies Lab, IT:U Interdisciplinary Transformation University, Linz, Austria.

*Corresponding author. Email: daniotti@csh.ac.at

text to a second LLM and request the model to generate a function on the basis of this description. Our use of two different LLMs—unlike previous approaches (29)—avoids creating unnecessarily strong correlations between human code and its transcription while ensuring that the (synthetic) AI-generated functions in our training data are close in functionality to the original human-written functions.

We then train a machine learning classifier on this dataset. Following Nguyen *et al.* (30), we transform each function using GraphCodeBert, a pretrained language model for code that embeds a function into a high-dimensional vector space using its tokens, comments, and the dataflow graph of its variables (31). The resulting vectors are fed into a classifier to determine whether a given function was written by a human or by genAI (Fig. 1, B and C).

Results

The classifier performs notably well, achieving an out-of-sample receiver operating characteristic (ROC) area under the curve (AUC) score of 0.96 (Fig. 1D) and an average rate of true positives of 0.95. We apply this classifier to 5 million functions extracted from 31 million contributions to Python projects from the beginning of 2019 to the end of 2024 for the full population of US-based users and the sampled users in the five other countries (Fig. 1D). In the supplementary materials (section S2), we show that the classifier also correctly identifies code generated by more recent LLMs introduced after our data collection ended as well as code produced in real-world interactions with LLMs, albeit with somewhat lower accuracy. Retraining the classifier on code produced by these newer LLMs further improves performance.

Figure 2A plots the AI adoption trajectory for US developers. Adoption rates sharply increase after major AI advancements, including the launches of Copilot, ChatGPT, and second-generation LLMs. Figure 2B compares the US with the five other major countries that we cover in the global race toward the adoption of AI. This shows that the US took an early lead, which it has managed to maintain ever since. About 29% of Python functions in the US were generated by AI by the end of 2024, closely followed by 23 and 24% for Germany and France, respectively. India draws close at 20%, after having initially lagged in adoption. By contrast, Russia and China have so far remained late adopters.

Focusing on the full population of US developers, we find that AI adoption rates drop with the number of years that developers have been active on GitHub. Figure 3B shows that whereas the most experienced developers use genAI in 27% of their code, developers who have just joined the GitHub platform use these tools for 37% of code. By contrast, using (self-reported) first name–based gender inference algorithms, we find no difference between men and women (Fig. 3A).

To assess how genAI affects the quantity and nature of code that developers produce, we rely on regression models with user and quarter-of-year fixed effects. This compares the output—in terms of quarterly number of commits—of the same developer at different points in time, controlling for economy-wide trends. These models, summarized in Fig. 3C, suggest a substantial impact of genAI on developer productivity. We find consistent effects across different sets of commits: all commits, commits that modify multiple files (which typically require navigating dependencies across scripts), and commits that add new libraries (which typically introduce new functionality) to scripts. Moving from 0 to 29% genAI usage—the estimated US adoption rate by the end of 2024—is associated with a 3.6% increase in commit rates across all these commit types. However, these associations with user productivity are fully driven by senior-level users, for whom a 29% adoption rate would imply a 6.2%

increase in commit rates (Fig. 3D). By contrast, we observe no statistically significant effects among early-career users.

Apart from increasing activity rates, AI adoption is also associated with increased experimentation with new libraries and combinations of libraries, which past research (32) has interpreted as signs of innovation. Because libraries often focus on specific types of functionality—such as visualization, natural language processing, web interactions, or database operations—these findings suggest that genAI helps developers expand their capabilities to new domains of software development. Our models predict that developers who use genAI at the average US end-of-2024 rate of 29% will implement 2.7% additional new combinations of libraries. Results are robust to variations in how we identify new library introductions. It is unlikely that the observed effects reflect the addition of esoteric libraries (“AI slop”): Findings do not change much if we only use the 5000 most common libraries or if we first group libraries into 124 coarse categories. Moreover, fig. S6 (supplementary materials, section S4.5) shows that these effects, as well as the earlier productivity effects, are likely lower bounds because errors in the measurement of users’ AI adoption rates bias each of these estimates downward.

Discussion

We set out to measure the use of genAI among software developers at the microlevel and on a global scale. Focusing on the software development labor force, we demonstrated how genAI has diffused and how this has affected the quantity and nature of code that developers produce. To do so, we developed a new genAI classifier to identify AI-generated functions in GitHub commits. Applied to a large dataset covering software development activity across six major countries, we document noticeable growth spikes in genAI-generated code soon after key genAI releases. Yet, we also observe notable differences between countries: The US leads, but its advantage is narrowing, with Germany and France close behind, India catching up fast, and China and Russia still lagging at the end of 2024. Corroborating existing studies (8, 9), our estimated adoption rates are higher among early-career developers. However, unlike most previous work, we find no significant differences between men and women.

We also find that genAI reshapes both the volume and nature of programming work. Using within-developer variation—comparing the same developer before and after adopting genAI—we show that AI adoption substantially increases output. Developers using genAI are also more likely to incorporate new combinations of software libraries into their code, which suggests that they venture into new technical domains (32) using unfamiliar building blocks (33). However, both productivity and exploration gains concentrate almost exclusively

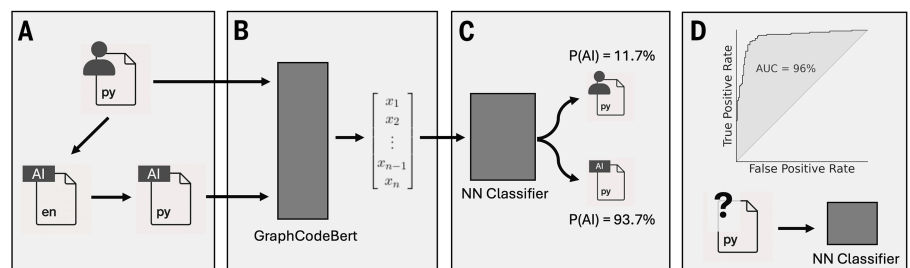


Fig. 1. Classifying code from functions written in the Python programming language as human or AI generated. (A) Using a collection of human-generated code, we ask one LLM to describe the code in English (en), then another to implement that description as a Python (py) function. (B) We vectorize the resulting code using GraphCodeBert, an embedding method that uses a code’s tokens, comments, and variable graph flow. (C) We train a neural network (NN) classifier combining GraphCodeBert with a classification head to predict the human versus AI labels. (D) We evaluate the classifier on out-of-sample data and apply it to a large database of unlabeled Python functions.

among senior-level developers. By contrast, although early-career developers used genAI more, they do not realize the same benefits.

Our observations that early-career developers use AI more but get less out of it compared with their more experienced colleagues may reflect differences in how well developers use genAI across a broader set of tasks. Senior-level developers will, for example, be quicker to interpret, and spot mistakes in, AI-generated code. Such an interpretation is supported by Hoffmann *et al.* (22), who show that access to genAI allows senior developers to spend less time on coordination activities and more time on coding.

Our findings both replicate and extend findings from other investigations. This study estimates the most recent adoption rates in the US at around 29%, which is very similar to adoption rates claimed for coding work at Microsoft (34) and Amazon (35). This shows that, despite our focus on code from open-source Python libraries, our results closely align with estimates of adoption rates from other contexts and may generalize beyond the specific setting of this study.

Unlike most other studies, our methodological advances and design allow us to compare early adoption rates across countries. We find a clear and sustained lead by US developers. Use of LLMs may be lower in countries such as China and Russia because of differences in their supply (providers such as OpenAI and Anthropic block access) and differences in demand [censorship limits local use, even though many users connect using virtual private networks (VPNs) (36)]. However, other major countries are quickly catching up, eroding the first-mover advantage of the US. Another feature that sets our study apart is that existing literature typically focuses on access to genAI—yielding reduced-form estimates of the causal effect of the so-called intention-to-treat, not of genAI itself—or usage in controlled experimental settings. By contrast, our approach allows us to quantify the intensity with which this technology is used in real-world work activities. Finally, we note that our cross-country evidence on genAI use complements firm-level survey work on broader AI adoption, which extends back to before the genAI era (18); although levels are not directly comparable, both perspectives document persistent cross-country gaps in AI use.

With respect to the productivity effects of genAI, our estimates are generally smaller than those found in RCTs (6, 37) and studies exploiting natural experiments (22, 24). In robustness checks (supplementary materials, section S4), we study whether nonlinearities or threshold effects in the benefits of genAI adoption can explain such discrepancies, but we find little evidence for this hypothesis. A more promising explanation is measurement error, which is likely to bias effect estimates downward. In line with this, fig. S6 (supplementary materials, section S4.5) shows that our effect estimates increase substantially if we correct them for measurement error. Moreover, we show that beneficial effects concentrate among senior developers, whereas early-career developers do not appear to benefit much from genAI. The higher effect estimates reported in prior studies may, therefore, also reflect differences in the populations and compiler samples that they analyzed. At the same time, the gap in benefits from genAI between senior and junior developers creates large uncertainties about the availability and nature of future career and learning paths for early-career developers.

This study has several limitations. First, our analysis focuses on software development. Although this limits its scope, work in this sector is specifically amenable to quantitative analysis at a level of granularity that is required to study how AI affects workers and their jobs. Within software, we focus only on Python-based open-source contributions. Although Python is a widely used language, adoption patterns may differ in other programming ecosystems. We argue that estimates derived from open-source Python code on GitHub are economically meaningful because open-source software (OSS) underpins most commercial stacks and generates substantial value (33, 38). GitHub's central role in collaboration, networking, and signaling further ties our evidence to professional activity (39, 40). Finally, that our estimates of AI use in the US align closely with reported AI use at leading US firms increases our confidence in the external validity of our findings.

More generally, we also do not account for potential externalities between co-workers or heterogeneity in productivity across firms, all of which may be relevant factors in how genAI affects programming activity. Beyond firms, our geographic analysis is limited to a subset of countries, and it would be important to widen the analysis to include countries at different income levels. In the specific case of China—where the programming community also relies on an alternative collaboration platform, Gitee (41)—there is some additional risk that our focus on GitHub projects distorts estimates. Finally, revisiting the effects of genAI, there are many other ways to evaluate the productivity of developers that heed more attention to code quality, from tracking how issues get resolved and code merges to implemented test coverage. Although feasible in principle, such analysis requires new data collection and careful design of metrics.

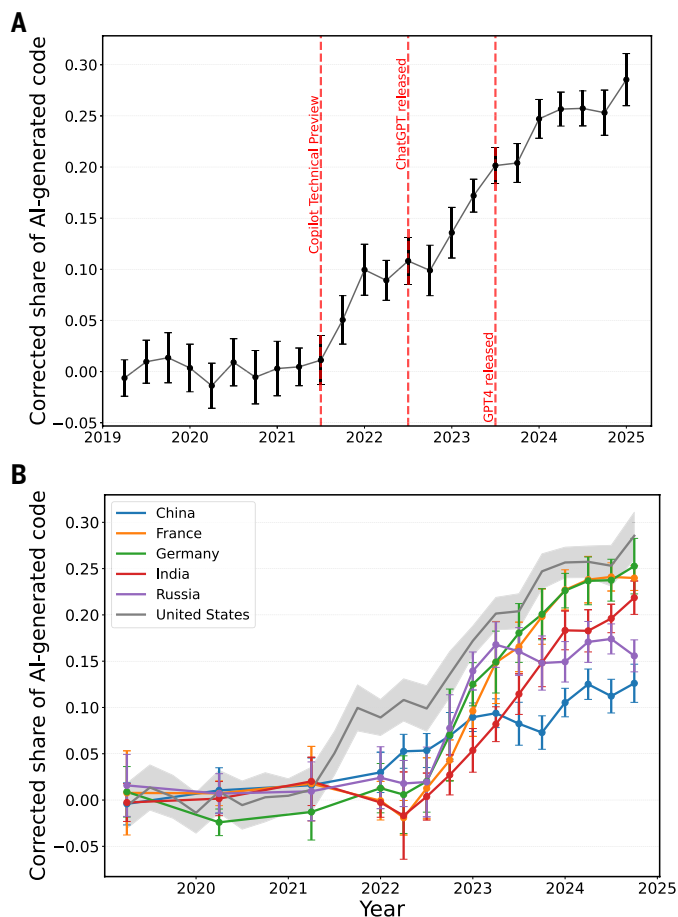


Fig. 2. Share of AI-generated Python functions over time. (A) Share of Python functions that were created or substantially changed by GitHub users in the US. Vertical lines depict 95% confidence intervals. The plot reveals abrupt shifts in adoption coinciding with key AI-related events: the release of GitHub Copilot Preview, the public launch of ChatGPT, and the second wave of LLM releases (GPT4 and related models). (B) Adoption in China, France, Germany, India, and Russia for which we sampled 2000 random developers per country-year [note that in China, GitHub competes with the alternative collaboration platform, Gitee (39)]. The US curve is replicated from (A) as a point of reference. The US led the early adoption of genAI, followed by European nations such as France and Germany. From 2023 onward, India rapidly catches up, whereas adoption in China and Russia progresses more slowly.

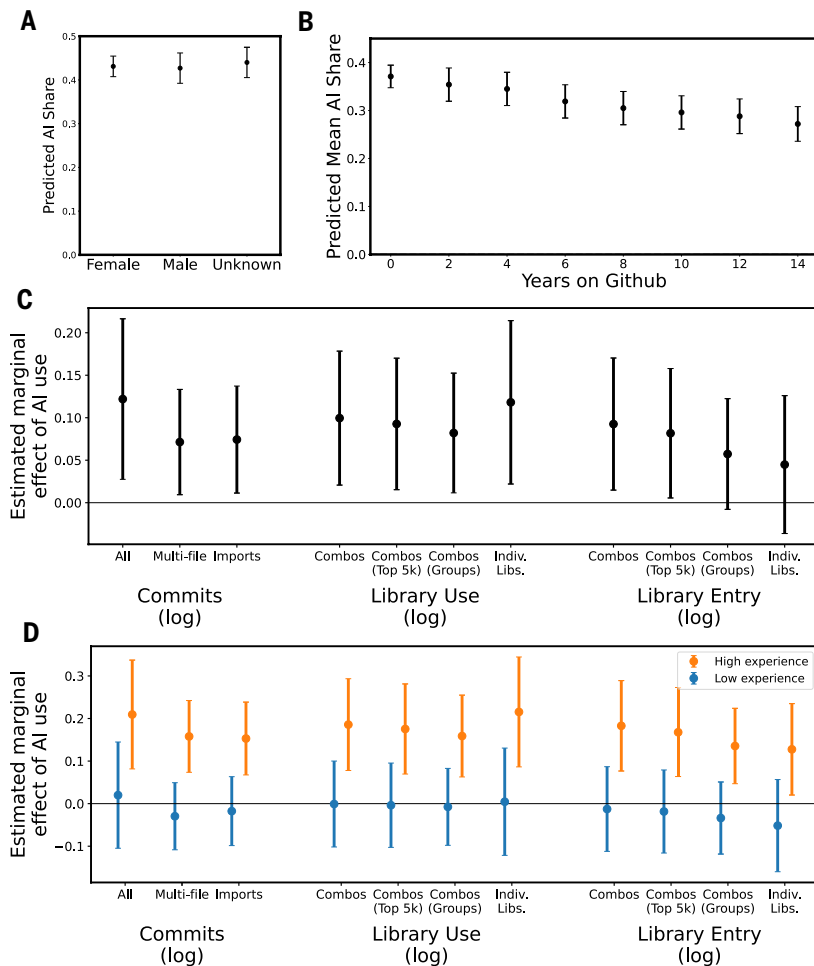


Fig. 3. Heterogeneity in adoption and effects. (A) Intensity of genAI use by gender inferred from GitHub user display names (US, 2024). (B) Intensity of genAI use by users' GitHub tenure (US, 2024). (C) Estimated effect of genAI use on user activity from a user-quarter panel regression with user and quarter fixed effects. GenAI use is associated with increased commit activity across all commits, multiple-file commits ("Multi-file") that navigate project interdependencies, and commits adding library imports ("Imports"), which we interpret as adding new features. GenAI is also associated with using wider ranges of libraries ("Indiv. Libs.") and of library combinations ("Compos") and increased experimentation with new libraries or combinations. Results are similar when subsetting on the 5000 most common library combinations ["Compos (Top 5k)"] and using coarsened library categories instead of libraries themselves ["Compos (Groups)"]. (D) However, these benefits accrue entirely to experienced developers, with no measurable gains for inexperienced developers. Error bars indicate 95% confidence intervals (standard errors are clustered by user).

We therefore leave questions about the effect of genAI on code quality for future research.

How much value has genAI created in coding? Although hard to answer definitively, our study offers some important pieces of this puzzle. Based on an analysis of detailed task surveys and wage statistics for about 900 different occupations, we estimate that the US spends between 637 billion and 1063 billion USD on labor costs related to coding activities per year (supplementary materials, section S6). Assuming that our estimated diffusion rates of 29% by the end of 2024 (based on open-source Python contributions) are representative of code in general, the annual value generated by genAI coding assistants in the US would depend on how much they increase productivity. Using our own conservative, baseline estimates, genAI would have increased the volume of commits by 3.6%. Assuming that these commits reflect valuable code contributions, our calculation implies that

genAI generates 23 billion to 38 billion USD of additional code per year. This estimate assumes that productivity gains are similar across programming languages. In a more conservative scenario, where productivity effects outside Python are negligible, the value of genAI would drop to about 17% of this figure (~4 billion to 6 billion USD), using estimates of Python's share of GitHub code (42).

By contrast, various laboratory experiments (21, 37) and field experiments (6) in software development have all yielded substantially larger causal effects of genAI on task completion times—arguably a more relevant quantity to track than commit volumes. Averaging across such studies (see materials and methods for details) yields an estimated 6.0 to 15.7% increase in productivity at a 29% adoption rate. This translates into a range of 38 billion to 167 billion USD for the direct impact of genAI on US coding activities. However, these estimates ignore that genAI may also lead to a reduction in the market price of code. Because this yields cost savings for consumers of code while reducing profits for suppliers (i.e., programmers and developers), factoring in such general equilibrium effects further widens the range of possible outcomes (supplementary materials, section S7). In the materials and methods, we show that this would mostly affect the upper bound of our estimates, with lower bounds all but unaffected. The upshot of these back-of-the-envelope calculations is that, although the total value of genAI to the US economy is uncertain, it is most likely substantial, on the order of at least tens of billions of US dollars.

Given that genAI has diffused quickly beyond the US, global cost savings would be larger still, even if we confine ourselves to the software sector. Moreover, we are currently still in the early phases of the diffusion curve of what looks to be a new general-purpose technology (3). Historically, early-stage productivity effects of general-purpose technologies have been hard to identify because it takes time to integrate them into firm-level workflows and procedures, train workers, and amass the complementary assets needed to fully exploit their potential. Because of this, we find ourselves on the bullish side of the debate when it comes to the productivity effects of genAI.

Our results on such effects and the heterogeneous diffusion of genAI raise important questions for policymakers and researchers. We need to understand the barriers to AI adoption: Are these barriers similar to prior radical innovations (43), or is this era different? Additionally, these barriers need to be understood not only at the individual level, but also at the firm, regional, and national levels. Our study takes a step toward answering such questions.

Moreover, given the wide dispersion in productivity across developers (44–46) and our finding that benefits accrue to more experienced coders only, future research should explore how AI adoption affects developer activity at the upper tail of elite developers, where the most important breakthroughs and innovations are likely to occur (47). Finally, our study exclusively focused on programming tasks. Yet, one study of elite software developers has proposed that access to genAI leads to a shift from managerial tasks to coding (22), which suggests that an important margin along which productivity effects materialize is shifts in the task composition of software developer jobs.

The nature of work often changes with the introduction of new technologies. Understanding these changes is especially difficult when the innovation in question is radical (43)—such as the spinning jenny,

transistors, or robots in the past—and at the same time pervasive (48). The uncertainty of the effects of genAI on work and labor markets is reflected in the wide range of attitudes that researchers and policymakers take toward it, ranging from utopian to skeptical and outright apocalyptic. These attitudes are formed in a fast-moving context and are based on incomplete evidence on the adoption and effects of AI. The findings in this study provide better evidence of how genAI is used in a large, important, and highly exposed sector of the economy as well as a way to monitor this in real time going forward. Applying our AI detection classifier to millions of code contributions made over a 6-year period, we can confirm that AI adoption is fast but heterogeneous across countries and individuals. Moreover, AI adoption is associated with increased activity rates in online software development collaborations.

However, one of the most unexpected findings was that genAI increased experimentation with new libraries. This suggests that genAI allowed users to advance faster to new areas of programming, embedding new types of functionality in their code. This corroborates prior findings (49) that genAI increases individual innovation, pushing individuals' capabilities in terms of the use of new combinations of libraries. However, again, only experienced, senior-level users seem able to leverage genAI in this way, with important consequences for career development and learning in the presence of genAI.

REFERENCES AND NOTES

1. F. Dell'Acqua, Navigating the Jagged Technological Frontier: Field Experimental Evidence of the Effects of AI on Knowledge Worker Productivity and Quality (SSRN, Harvard Business School Technology and Operations Management Unit, working paper no. 24-013, 2023); <https://ssrn.com/abstract=4573321>.
2. E. Mollick, *Co-Intelligence: Living and Working with AI* (Penguin, 2024).
3. T. Eloundou, S. Manning, P. Mishkin, D. Rock, *Science* **384**, 1306–1308 (2024).
4. E. Brynjolfsson, *Commun. ACM* **36**, 66–77 (1993).
5. S. Noy, W. Zhang, *Science* **381**, 187–192 (2023).
6. Z. Cui *et al.*, The Effects of Generative AI on High-Skilled Work: Evidence from Three Field Experiments with Software Developers (SSRN, 2025); <https://ssrn.com/abstract=4945566>.
7. T. Teubner, C. M. Flath, C. Weinhardt, W. van der Aalst, O. Hinz, *Bus. Inf. Syst. Eng.* **65**, 95–101 (2023).
8. A. Bick, A. Blandin, D. J. Deming, "The rapid adoption of generative AI" (National Bureau of Economic Research, working paper 32966, 2025); https://www.nber.org/system/files/working_papers/w32966/w32966.pdf.
9. A. Humlum, E. Vestergaard, *Proc. Natl. Acad. Sci. U.S.A.* **122**, e2414972121 (2025).
10. D. Acemoglu, *Econ. Policy* **40**, 13–58 (2025).
11. A. Humlum, E. Vestergaard, Large Language Models, Small Labor Market Effects (SSRN, University of Chicago, Becker Friedman Institute for Economics, working paper no. 2025-56, 2025); <https://ssrn.com/abstract=5219933>.
12. E. Andreadis, E. Chatzikonstantinou, E. Kalotychou, C. Louca, C. Makridis, Local Heterogeneity in Artificial Intelligence Jobs Over Time and Space (SSRN, 2024); <https://ssrn.com/abstract=5078805>.
13. D. Bearson, N. Wright, Strategic Targeting and Unequal Global Adoption of Artificial Intelligence (SSRN, Columbia Business School, research paper no. 5187851, 2025); <https://ssrn.com/abstract=5187851>.
14. D. Comin, B. Hobijn, *Am. Econ. Rev.* **100**, 2031–2059 (2010).
15. P. A. David, *Am. Econ. Rev.* **80**, 355–361 (1990).
16. E. Brynjolfsson, D. Rock, C. Syverson, *Am. Econ. J. Macroecon.* **13**, 333–372 (2021).
17. R. Juhász, M. P. Squicciarini, N. Voigtländer, *J. Polit. Econ.* **132**, 3215–3259 (2024).
18. F. Calvino, L. Fontanelli, "A portrait of AI adopters across countries: Firm characteristics, assets' complementarities and productivity" (OECD Science, Technology and Industry Working Papers, no. 2023/02, 2023); <https://doi.org/10.1787/0fb79bb9-en>.
19. Y. Ling, A. Kale, A. Imas, Underreporting of AI Use: The Role of Social Desirability Bias (SSRN, 2025); <https://ssrn.com/abstract=5464215>.
20. D. Almog, *arXiv:2511.18582* [econ.GN] (2025).
21. S. Peng, E. Kalliamvakou, P. Cihon, M. Demirel, *arXiv:2302.06590* [cs.SE] (2023).
22. M. Hoffmann, S. Boysel, F. Nagle, S. Peng, K. Xu, Generative AI and the Nature of Work (SSRN, Harvard Business School Strategy Unit, working paper no. 25-021, 2025); <https://ssrn.com/abstract=5007084>.
23. D. Yeverechyahu, R. Mayya, G. Oestreicher-Singer, *arXiv:2409.08379* [cs.SE] (2024).
24. F. Song, A. Agarwal, W. Wen, *arXiv:2410.02091* [cs.SE] (2024).
25. S. Aum, Y. Shin, "Is software eating the world?" (National Bureau of Economic Research, working paper 32591, 2024); https://www.nber.org/system/files/working_papers/w32591/w32591.pdf.
26. S. Juhász, J. Wachs, J. Kaminski, C. A. Hidalgo, *arXiv:2407.13880* [econ.GN] (2024).
27. K. Handa *et al.*, *arXiv:2503.04761* [cs.CV] (2025).
28. R. M. Del Rio-Chanona, N. Laurentsyeva, J. Wachs, *PNAS Nexus* **3**, pgae400 (2024).
29. T. Ye *et al.*, *arXiv:2405.16133* [cs.SE] (2024).
30. P. T. Nguyen *et al.*, *J. Syst. Softw.* **214**, 112059 (2024).
31. D. Guo *et al.*, *arXiv:2009.08366* [cs.SE] (2021).
32. H. Fang, J. Herbsleb, B. Vasilescu, in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering* (Association for Computing Machinery, 2024), pp. 1–11.
33. N. Eghbal, "Roads and Bridges: The Unseen Labor Behind Our Digital Infrastructure" (Ford Foundation, 2016); <https://www.fordfoundation.org/learning/library/research-reports/roads-and-bridges-the-unseen-labor-behind-our-digital-infrastructure/>.
34. J. Novet, "AI now writes a big chunk of code at Microsoft and Google — And it could be coming for even more at Meta," *Business Insider*, 30 April 2025; <https://www.businessinsider.com/ai-code-meta-microsoft-google-llamacon-engineers-2025-4>.
35. N. Scheiber, "At Amazon, some coders say their jobs have begun to resemble warehouse work," *New York Times*, 27 May 2025; <https://www.nytimes.com/2025/05/25/business/amazon-ai-coders.html>.
36. H. Bao, M. Sun, M. Teplitskiy, *Quant. Sci. Stud.* **6**, 716–731 (2025).
37. E. Paradis *et al.*, in *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)* (IEEE, 2025), pp. 618–629.
38. M. Hoffmann, F. Nagle, Y. Zhou, The Value of Open Source Software (SSRN, Harvard Business School Strategy Unit, working paper no. 24-038, 2024); <https://ssrn.com/abstract=4693148>.
39. L. Dabbish, C. Stuart, J. Tsay, J. Herbsleb, in *Proceedings of the ACM 2012 Conference on Computer Supported Cooperative Work* (Association for Computing Machinery, 2012), pp. 1277–1286.
40. L. Abou El-Komboz, M. Goldbeck, *Labour Econ.* **97**, 102800 (2025).
41. J. Gortmaker, "Open source software policy in industry equilibrium" (2024); https://jeffgortmaker.com/files/Open_Source_Software_Policy_in_Industry_Equilibrium.pdf.
42. GitHub 2.0, GitHub language stats (2025); https://madnight.github.io/github/#/pull_requests/2024/1.
43. K. Frenken, M. B. Punt, *SocArXiv* (OSF, 2023); https://osf.io/preprints/socarxiv/6cr5t_v1.
44. H. Sackman, W. J. Erikson, E. E. Grant, *Commun. ACM* **11**, 3–11 (1968).
45. G. E. Bryan, in *1994 IEEE Aerospace Applications Conference Proceedings* (IEEE, 1994), pp. 55–62.
46. L. Betti, L. Gallo, J. Wachs, F. Battiston, *Nat. Commun.* **16**, 3956 (2025).
47. H. Fang, P. Park, J. Evans, J. Herbsleb, B. Vasilescu, *arXiv:2411.05646* [cs.SE] (2024).
48. C. Freeman, C. Perez, in *Technical Change and Economic Theory*, G. Dosi, C. Freeman, R. Nelson, G. Silverberg, L. Soete, Eds. (Pinter, 1988), pp. 38–66.
49. A. R. Doshi, O. P. Hauser, *Sci. Adv.* **10**, eadn5290 (2024).

ACKNOWLEDGMENTS

We thank U. Schetter, H. Vipond, A. Musso, and participants of the ANETI Brownbag seminar for helpful comments. We thank M. Salamon for valuable research assistance. **Funding:** This study was supported by the Austrian Research Promotion Agency (FFG), grant ESSENCE (873927) (S.D., J.W., X.F., and F.N.), and the Hungarian National Scientific Fund, grant OTKA FK-145960 (J.W.). **Author contributions:** Conceptualization: S.D., F.N.; Data analysis: S.D., J.W., X.F., F.N.; Data collection: S.D., J.W.; Data collection and estimation, US wage sums: X.F.; Primary method: S.D.; Writing – original draft: F.N., J.W.; Writing – review & editing: S.D., J.W., X.F., F.N. **Competing interests:** The authors declare no competing interests. **Data, code, and materials availability:** Data to replicate our analyses are available through Dryad (72). **License information:** Copyright © 2026 the authors, some rights reserved; exclusive licensee American Association for the Advancement of Science. No claim to original US government works. <https://www.science.org/about/science-licenses-journal-article-reuse>

SUPPLEMENTARY MATERIALS

science.org/doi/10.1126/science.adz9311
Materials and Methods; Supplementary Text; Figs. S1 to S9; Tables S1 to S18; References (50–71)

Submitted 19 June 2025; accepted 29 December 2025; published online 22 January 2026

10.1126/science.adz9311



Who is using AI to code? Global diffusion and impact of generative AI

Simone Daniotti, Johannes Wachs, Xiangnan Feng, and Frank Neffke

Science **391** (6787), . DOI: 10.1126/science.adz9311

Editor's summary

As generative artificial intelligence (genAI) advances reshape careers, one profession remains distinctly affected: software development. Recent research found that genAI job cuts eliminated many entry-level positions, whereas senior-level developer jobs, which require critical thinking and uncoded, tacit skills, were spared. However, a fuller picture was needed to understand genAI's impact on human innovation and inequities. Daniotti *et al.* trained a machine learning classifier to detect genAI-generated code among software developers in the US, China, France, Germany, India, and Russia (see the Perspective by Wu and Vasilescu). Although entry-level developers used genAI the most, it did not appear to benefit them. Experienced, senior-level developers, by contrast, leveraged genAI to increase their productivity and software innovations. The growing skills gap may be a bellwether for changes in genAI-exposed industries. —Ekeoma Uzogara

View the article online

<https://www.science.org/doi/10.1126/science.adz9311>

Permissions

<https://www.science.org/help/reprints-and-permissions>

Use of this article is subject to the [Terms of service](#)

Science (ISSN 1095-9203) is published by the American Association for the Advancement of Science. 1200 New York Avenue NW, Washington, DC 20005. The title *Science* is a registered trademark of AAAS.

Copyright © 2026 The Authors, some rights reserved; exclusive licensee American Association for the Advancement of Science. No claim to original U.S. Government Works